

# Les tuples

- Vous savez qu'il n'est pas possible de changer les caractères au sein d'une chaîne existante, alors que vous pouvez modifier les éléments d'une liste.
- Un tuple est une collection d'éléments à l'intérieur de parenthèses séparés par des virgules semblable à une liste mais il n'est pas modifiable.
- Un tuple peut représenter une clé de dictionnaire ce qu'une liste ne peut faire.
- Les tuples sont préférables aux listes partout où l'on veut être certain que les données transmises ne soient pas modifiées par erreur.

```
>>> saisons = ("printemps", "été", "automne", "hiver")
>>> print(saisons)
('printemps', 'été', 'automne', 'hiver')
>>> print(saisons[2:3])
('automne',)
>>> print(("printemps",) + saisons[1:4])
('printemps', 'été', 'automne', 'hiver')
>>> saisons[2] = "Automne"
```

Il faut toujours au moins une virgule pour définir un tuple.

Traceback (most recent call last):

File "<pyshell#4>", line 1, in <module>

saisons[2] = "Automne"

TypeError: 'tuple' object does not support item assignment

## ■ Création et affectation d'un tuple (suite)

```
>>> un_tuple = ()  
>>> print un_tuple  
( )  
>>> tuple("abc")  
( 'a', 'b', 'c' )
```

tuple vide

Pour accéder aux valeurs d'un tuple, on utilise les crochets de la même façon qu'avec les listes.

```
>>> Couleur = ("blanc", "bleu", "rouge", "mauve", "noir")  
>>> print(Couleur[1:3])  
( 'bleu', 'rouge' )  
>>> print(Couleur[:3])  
( 'blanc', 'bleu', 'rouge' )  
>>> print(Couleur[3:])  
( 'mauve', 'noir' )
```

## ■ Modification d'un tuple

Cela ne peut se faire directement. Il faut créer un nouveau tuple à l'aide d'anciens grâce à l'opérateur de concaténation ou de répétition.

```
>>> Teinte = Couleur[1], Couleur[2], Couleur[4]
>>> print(Teinte)
('bleu', 'rouge', 'noir')
>>> Teinte = Teinte[0:2] + ("vert",)
>>> print(Teinte * 2)
('bleu', 'rouge', 'vert', 'bleu', 'rouge', 'vert')
```

On peut modifier certains éléments d'un tuple si ces éléments sont modifiables.

```
>>> T = (5, [0, 1, 2])
>>> print (T)
(5, [0, 1, 2])
>>> T[0] = 6
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    T[0] = 6
TypeError: 'tuple' object does not support item assignment
>>> T[1][0] = -1
>>> print (T)
(5, [-1, 1, 2])
```

## ■ Suppression d'un tuple ou des éléments d'un tuple

Il est impossible de supprimer individuellement les éléments d'un tuple.  
On peut assembler un nouveau tuple en omettant les éléments indésirables.  
Pour supprimer explicitement un tuple entier, on utilise del :  
del (teinte)

■ Les opérateurs \*, +, in, [], <, >, ==, <=, >= et les fonctions intégrées len, max, min se comportent exactement de la même façon avec les tuples qu'avec les listes.

■ Les méthodes de manipulation de listes : tri, remplacement, insertion, etc. – ne sont pas implémentées.

■ Un ensemble d'éléments séparés par des virgules et écrits sans symboles d'identification de type crochets pour les listes, parenthèses pour les tuples, etc. est par défaut un tuple.

```
>>> "Abc", 99
('Abc', 99)
>>> x = 23, 45, "q"
>>> x
(23, 45, 'q')
```

```
>>> u, v = 0, 1
>>> u, v
(0, 1)
>>> print (u, v)
0 1
```

■ Les fonctions intégrées list() et tuple() permettent de convertir une liste en tuple ou vice versa.

```
>>> T = (2, 3)
>>> T = list(T)
>>> print (T)
[2, 3]
```